

Low-Level Design

1. Module-Level Design

Document Processing Module

``PDFKnowledgeBaseProcessor`` reads the PDF, extracts page text, and emits chunk-ready content. The module preserves source filename and page number for downstream citation.

Chunking Module

Chunking is implemented inside the ingestion module using a fixed-size sliding window with overlap. The algorithm prefers sentence boundaries when possible before cutting a chunk.

Embedding Module

``HashingEmbedder`` converts text into deterministic fixed-length vectors. It exposes ``embed_documents()`` and ``embed_query()`` so the rest of the system stays independent from the actual embedding implementation.

Vector Storage Module

``ChromaKnowledgeStore`` wraps ChromaDB. It exposes:

- ``index_chunks(chunks)`` to upsert embeddings and metadata
- ``query(query, k)`` to return scored retrieved chunks

Retrieval Module

Retrieval is handled by the vector store wrapper. It converts a user query into an embedding, performs top-k lookup, and maps ChromaDB output into application-level ``RetrievedChunk`` objects.

Query Processing Module

``IntentRouter`` classifies the query and determines whether the answer path or HITL path should be taken based on confidence and risk signals.

Graph Execution Module

``SupportWorkflow`` builds a LangGraph state graph with three operational nodes:

- ``process_query``
- ``human_review``
- ``output_response``

This extends the required 2-node core flow by inserting an optional HITL checkpoint.

HITL Module

``HumanEscalationService`` creates escalation tickets and stores human resolutions in append-only JSONL logs.

2. Data Structures

Document Representation

The PDF is represented as extracted page text plus source metadata:

- `source`
- `page\_number`
- `text`

Chunk Format

Each chunk is stored as:

- `chunk\_id`
- `text`
- `source`
- `page\_number`
- `chunk\_index`

Embedding Structure

Embeddings are numeric float arrays with a fixed dimension. In this implementation the vector dimension is 256.

Query-Response Schema

`SupportResult` contains:

- `query`
- `intent`
- `route`
- `confidence`
- `answer`
- `citations`
- `escalation\_reason`
- `ticket\_id`

Graph State Object

The `SupportState` object carries:

- `query`
- `intent`
- `route`
- `retrieved\_chunks`
- `context\_text`

- `confidence`
- `answer`
- `citations`
- `escalation\_reason`
- `ticket\_id`
- `human\_response`
- `final\_response`

3. Workflow Design (LangGraph)

Nodes

- `process\_query`: detects intent, retrieves chunks, scores confidence, drafts answer
- `human\_review`: creates a ticket or integrates a human override
- `output\_response`: returns the final answer payload

Edges

- `START -> process\_query`
- `process\_query -> output\_response` when route is `answer`
- `process\_query -> human\_review` when route is `hitl`
- `human\_review -> output\_response`
- `output\_response -> END`

State

State is a mutable dictionary-like object shared across nodes. Each node enriches the state instead of reconstructing the full object from scratch.

4. Conditional Routing Logic

Answer Generation Criteria

The assistant answers directly when:

- relevant chunks are retrieved
- the top retrieval score is above the confidence threshold
- the query does not contain sensitive escalation keywords
- the request appears focused on a single issue

Escalation Criteria

Escalation is triggered when:

- no relevant chunks are found
- the retrieval score is low

- the query contains sensitive or high-risk language
- the request combines multiple issues that are likely to need judgment

5. HITL Design

When Escalation Is Triggered

The workflow routes to ``human_review`` whenever the router determines the case is low-confidence or sensitive.

What Happens After Escalation

The system writes an escalation record with a generated ticket ID, timestamps, a reason for escalation, and a short context preview.

How Human Response Is Integrated

If a support specialist provides a response later, the workflow can be re-run with the original ticket ID and a ``human_response`` payload. The HITL node then logs the resolution and emits the human-approved answer as the final output.

6. API / Interface Design

Input Format

CLI commands accept:

- a PDF path for ingestion
- a plain-text query for single-turn asking
- optional human-review confirmation in interactive mode

Output Format

The assistant returns:

- answer text
- intent
- route
- confidence
- citations
- optional ticket ID

Interaction Flow

1. Run ingestion once per knowledge base update.
2. Submit queries through ``ask`` or ``chat``.
3. If escalation occurs, inspect the created ticket or provide a manual override in ``--auto-hitl`` mode.

7. Error Handling

Missing Data

If the PDF contains no extractable text, the ingestion stage returns zero chunks and the assistant later escalates missing-context queries.

No Relevant Chunks Found

The router sends the query to HITL with a clear reason instead of fabricating an answer.

LLM Failure

The current implementation uses an offline synthesizer, but the module boundary anticipates external LLM integration. In production, LLM exceptions should be caught and converted into a controlled fallback or escalation response.

Workflow Diagram

